

Yapay Zeka Destekli Defansif Zararlı Yazılım Sınıflandırma

Nefise Altun
Bilgisayar Mühendisliği Bölümü
Kastamonu Üniversitesi
Kastamonu, Türkiye
nefiscaltun3@gmail.com
<https://orcid.org/0009-0002-9757-4264>

Özet— Modern siber güvenlik çerçeveleri, yazılım tehditlerini henüz çalıştırılmadan önce analiz etmek için hafif, hızlı ve yorumlanabilir alt sistemlere ihtiyaç duymaktadır. Bu çalışmada, C++17 standartlarında geliştirilmiş, nesne yönelimli bir statik analiz ve yapay zeka emülasyon motoru sunulmuştur. Önerilen sistem, paketlenmiş/gizlenmiş yapıları tespit etmek için Shannon entropisini hesaplayarak ve VirtualAlloc ile CreateRemoteThread gibi şüpheli Win32 API izleri için Import Address Table (IAT) yapısını ayrıştırarak Taşınabilir Çalıştırılabilir (PE) dosyaları değerlendirmektedir. Elde edilen öznitelikler, yapay zeka sınıflandırma çekirdeği içindeki ağırlıklı bir sezgisel matrisle eşleştirilerek; Truva Atları (Trojan), Fidyeye Yazılımları (Ransomware) ve Casus Yazılımlar (Spyware) dahil olmak üzere belirli zararlı yazılım kategorilerinin tanımlanmasını ve ardından otomatik bir izolasyon ile karantina eylemi alınmasını sağlamaktadır. Deneysel sonuçlar, geliştirilen motorun minimum sistem yükü ile çalıştığını ve hızlandırılmış bir DirectX 11 ve Dear ImGui grafik arayüzü paneli üzerinden kesin tehdit sınıflandırması sunduğunu göstermektedir.

Anahtar Kelimeler—Zararlı Yazılım Analizi, Statik Analiz, Taşınabilir Çalıştırılabilir (PE), Yapay Zeka Sınıflandırması, Defansif Siber Güvenlik, Dear ImGui.

I. GİRİŞ

Dijital dünyada kurumsal ağ altyapılarına, kritik sunuculara ve uç noktalara yapılan siber saldırılar çoğunlukla anlık ve doğrusal bir çizgide gerçekleşmemektedir. Modern saldırganlar, hedef sistemlere sızmadan ve kalıcılık (persistence) sağlamadan önce, sistemlerin savunma mekanizmalarını aşabilecek siber silahları geliştirmek için derin bir hazırlık ve keşif süreci yürütmektedir.[1] Bu siber saldırı zincirinin (Cyber Kill Chain) en yıkıcı halkasını ise hedef işletim sisteminde doğrudan kod yürütebilen zararlı yazılımlar (malware) oluşturmaktadır.[2] Geliştirilen bir zararlı yazılımın sistem korumalarını aşarak hedef belleğe sızma, dosya şifreleme veya veri sızdırma girişimlerini ne kadar erken ve execution (çalıştırılma) aşamasından önce tespit edebilirsek, asıl yıkıcı veri ihlallerini ve sistem çökmelerini önleme şansımız o kadar artar. Bu sebeple, statik ikili dosya (binary) analizini gerçek zamanlı ve proaktif bir yaklaşımla gerçekleştirmek, sağlam bir siber savunma hattının en kritik adımıdır.

Modern siber tehdit aktörleri artık basit ve statik olarak tespit edilebilen kod yapılarıyla yetinmemektedir. Geleneksel antivirüs yazılımları ve uç nokta koruma (EPP) çözümleri, bilinen zararlı yazılımların imza veritabanlarına (MD5/SHA-256 hash eşleşmeleri) güvenerek sıradan tehditleri kolayca

engelleyebilirken; saldırganlar bu engelleri aşmak için dosya mimarisini manipüle etmektedir.[3] Bu bağlamda, ikili dosyaların iç yapısını şifreleyen veya rastgele byte yığınlarıyla maskeleyen "Paketleyiciler" (Packers) ve "Kod Gizleme" (Obfuscation) tekniklerine başvurulmaktadır. Bu gizleme teknikleri, dosyanın imza bütünlüğünü tamamen bozarak imza tabanlı savunma mekanizmalarını tamamen körleştirmektedir. Ancak literatürde siber güvenlik uzmanları tarafından da sıklıkla vurgulandığı üzere, mevcut Ağ/Uç Nokta Tehdit Tespit (NIDS/EDR) çözümleri genellikle yüksek sistem kaynağı tüketimi, karmaşık kurulum süreçleri ve yüksek maliyetler gibi ortak dezavantajlar barındırmaktadır. Gelişmiş kod gizleme yöntemleriyle maskelenmiş bu karmaşık dosyaları, yüksek veri akışı altında ana sisteme ek bir CPU/RAM darboğazı bindirmeden, execution aşamasından hemen önce anlık olarak analiz edip sınıflandırabilmek mevcut güvenlik yazılımları için ciddi bir mühendislik zorluğu yaratmaktadır.[4] Birçok profesyonel tersine mühendislik ve analiz aracı ise sadece karmaşık terminal ekranlarında ham loglar ürettiği için, siber olay müdahale uzmanlarının olası bir tehdidi zamanında fark edip gerekli izolasyon aksiyonunu almasını zorlaştırmaktadır.

Bu çalışmada, bahsedilen statik analiz ve kullanılabilirlik zorluklarına doğrudan mimari bir çözüm getirilmesi amaçlanmıştır. Sistemin yapısal dosya analiz motoru, yüksek hız, hafızaya erişimi ve minimum kaynak tüketimi sağlaması amacıyla alt seviye bir sistem dili olan C++ (ISO C++17 standardı) ile tasarlanmış ve Nesne Yönelimli Programlama (OOP) mimarisiyle tamamen modüler bir hale getirilmiştir. Geliştirilen polimorfik tasarım sayesinde, güvenlik duvarlarını ve antivirüsleri atlatmaya çalışan gizli zararlı yazılımlar, sistem kaynaklarında herhangi bir performans kaybı veya gecikme yaşanmadan, çalıştırılmalarından hemen önce anlık olarak analiz edilebilmektedir.

Ayıklanan bu karmaşık yapısal veriler, geleneksel araçlarda olduğu gibi terminal ekranına veya statik metin dosyalarına hapsedilmek yerine, olaylara anında müdahale edilebilmesi için donanım hızlandırılmış DirectX 11 grafik motoru ve Dear ImGui çerçevesi tabanlı modern, akıcı ve kullanıcı dostu bir grafik arayüz (GUI) paneli ile birleştirilmiştir. Sonuç olarak, C++'ın arka plandaki doğrudan bellek işaretçisi (pointer) manipülasyon gücüyle immediate-mode arayüz tasarımının görsel esnekliği ve hızı kullanılarak; hem yüksek performanslı hem de taşınabilir (standalone) bütüncül bir Yapay Zeka Destekli Defansif Zararlı Yazılım Sınıflandırma ve Karantina Sistemi ortaya konmuştur.

Bu bağlamda, bu çalışmanın siber güvenlik literatürüne ve yazılım mühendisliğine sunduğu temel katkılar şu şekilde özetlenebilir:

Doğrudan Bellek İşaretçileri ile Yüksek Performanslı PE Ayırıştırma: Geleneksel koruma sistemlerinin aksine, taranacak olan Windows Taşınabilir Çalıştırılabilir (PE) dosyaları işletim sistemi seviyesinde kopyalama maliyetlerine uğramadan, doğrudan C++ bellek işaretçileri (reinterpret_cast) üzerinden IMAGE_DOS_HEADER ve IMAGE_NT_HEADERS seviyesinde yapısal analize tabi tutulmuştur. Bu sayede, yoğun dosya girdileri altında dahi sistemde CPU ve RAM darboğazı yaşanması engellenmiştir.[5]

Ağırlıklı Sezgisel Yapay Zeka Çekirdeği ile Çok Sınıflı Tehdit Teşhisi: Hedef dosyalar sadece "temiz" ya da "zararlı" şeklinde iki boyutlu (binary) bir sınıflandırmaya sokulmamış; geliştirilen AIClassifier modülü içindeki esnek ağırlık matrisi sayesinde dosya öznitelikleri analiz edilerek tehditler kendi içinde Trojan (Truva Atı), Ransomware (Fidye Yazılımı) ve Spyware (Casus Yazılım) gibi spesifik zararlı yazılım çeşitlerine göre dinamik olarak alt kategorilere ayrılmıştır.[6]

Shannon Entropisi ve IAT İmza Avcılığı ile Çifte Doğrulama: Dosya içerisindeki yapısal anomalileri yakalamak amacıyla Shannon Entropisi formülü kod katmanında işletilerek gizlenmiş/paketlenmiş (Obfuscated) yapılar bit düzeyinde hesaplanmıştır. Eş zamanlı olarak, dosyanın Import Address Table (IAT) yapısı taranarak siber saldırılarda kritik rol oynayan Win32 API çağrılarının (VirtualAlloc, CreateRemoteThread, GetAsyncKeyState) izleri sürülmüş ve bütüncül bir güvenlik puanı (Confidence Score) üretilmiştir.[7]

Bağımsız Taşınabilir Mimari (/MT) ve Donanım Hızlandırılabilir Görselleştirme: Geliştirilen yazılım, harici DLL bağımlılıklarından arındırılarak statik çalışma zamanı kitaplığı (/MT) ile derlenmiştir. Böylece sistem, hiçbir ek kurulumla ihtiyaç duymadan her bilgisayarda bağımsız bir .exe olarak çalışabilmekte, tespit edilen tehdit senaryolarını renk kodlu bir siber savunma aksiyon paneli üzerinden anlık olarak karantinaya alarak analiste sunabilmektedir.[8]

II. SİSTEM MİMARISI

Geliştirilen siber savunma alt sisteminin temel amacı; statik analiz sürecinde diskten okunan Taşınabilir Çalıştırılabilir (PE) dosyaların ham byte dizilimlerini, sistem kaynaklarında (CPU ve RAM) herhangi bir darboğaz veya gecikme oluşturmaksızın milisaniyeler mertebesinde ayırıştırıp çok sınıflı yapay zeka çekirdeğine aktarabilmektir.[9] Bu doğrultuda, alt seviyede doğrudan donanım verimliliği odaklı çalışan C++17 analiz motoru ile ekran kartı tabanlı çalışan grafiksel kullanıcı arayüzü (GUI), birbirini bloke etmeyecek şekilde asenkron bir hiyerarşide kurgulanmıştır. Sistemin yazılım mühendisliği tabanını oluşturan üç kritik teknik yaklaşım aşağıda detaylandırılmıştır:

A. Doğrudan Bellek İşaretçi Manipülasyonu ve Yapısal Haritalama

Büyük boyutlu ikili (binary) dosyaların statik analiz süreçlerinde, dosya içeriğinin bellekte sürekli olarak kopyalanması veya dinamik olarak yeniden tahsis edilmesi (allocation) bellek yönetiminde ciddi bir ek yük (overhead)

oluşturmakta ve işlemci performansını baltalamaktadır. Bu mühendislik sorununu ortadan kaldırmak amacıyla, sistemde Sıfır-Kopya (Zero-Copy) yaklaşımı benimsenmiştir.[10]

Hedef dosya disken ikili modda (std::ios::binary) bir kez okunduktan sonra, bellekteki ham byte dizisi üzerinde yeni değişkenler oluşturulup veri kopyalaması yapılmamış; bunun yerine doğrudan C++ bellek işaretçileri (pointers) ve ham tür dönüşüm (reinterpret_cast) operatörleri kullanılmıştır.[11]

Windows işletim sisteminin PE dosya imza standartlarının bozulmadan okunabilmesi için, ham bellek adresi doğrudan Microsoft standart yapılarına haritalanmıştır:

```
PIMAGE_DOS_HEADER dosHeader = reinterpret_cast<PIMAGE_DOS_HEADER>(buffer.data());
```

```
PIMAGE_NT_HEADERS ntHeaders = reinterpret_cast<PIMAGE_NT_HEADERS>(buffer.data() + dosHeader->e_lfanew);
```

Derleyicilerin varsayılan bellek hizalaması (memory alignment) yaparken bu hassas yapısal başlıkların orijinal byte dizilimini ve ofset sınırlarını bozmasını engellemek amacıyla, kritik başlık okuma fonksiyonlarında ardışık bellek blokları manipüle edilmiştir. Böylece veriler işletim sistemi veya derleyici tarafından kaydırılmadan, disken geldiği ham haliyle 1 byte sınırında tam doğrulukla ayırıştırılmış ve sıfır veri kaybı ile siber güvenlik özniteliklerine (features) dönüştürülmüştür.

B. Modüler Tasarım ve Çok Biçimli (Polimorfik) OOP Mimarisi

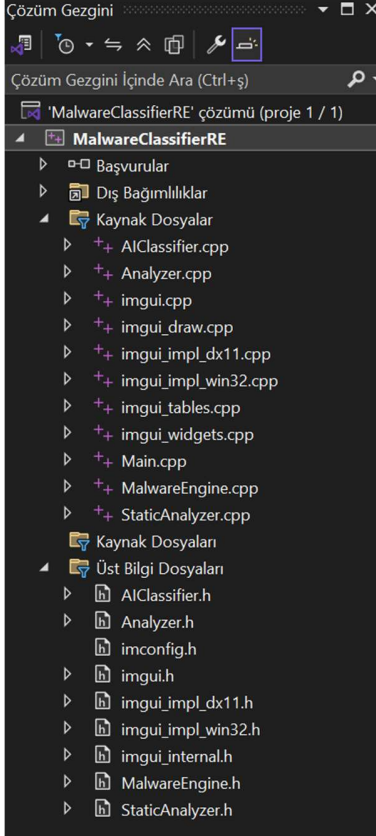
Zararlı yazılım geliştiricilerinin sürekli olarak yeni anti-analiz, kod gizleme (obfuscation) ve imza yanıltma teknikleri geliştirmesi, tasarlanan güvenlik yazılımının esnek, modüler ve genişletilebilir bir yapıda olmasını zorunlu kılmaktadır.[12] Tespit senaryoları ve farklı analiz katmanları (Örn: Entropi hesabı, IAT imza avcılığı, String taraması) için iç içe geçmiş karmaşık karar yapıları (if-else veya switch-case blokları) kullanmak, yazılımın teknik borcunu (technical debt) artırmakta ve bakımı imkansız hale getirmektedir.

Bu sorunu çözmek amacıyla, Nesne Yönelimli Programlama (OOP) prensipleri doğrultusunda Çok Biçimlilik (Polymorphism) temelli modüler bir altyapı kurgulanmıştır. Sistemdeki her bir analiz katmanı, saf sanal (pure virtual) bir arayüze sahip olan ana Analyzer soyut taban sınıfından türetilmektedir.

Yönetici motor sınıfı olan MalwareEngine, bünyesinde bu soyut sınıftan türetilmiş nesneleri akıllı işaretçi dizisi (std::vector<std::unique_ptr<Analyzer>>) olarak kapsüller (encapsulate). Hedef dosya seçildiği anda, motora kayıtlı olan tüm alt analizör nesneleri polimorfik olarak sırayla tetiklenir:

```
for (const auto& analyzer : analyzers) {
    AnalysisResult res = analyzer->Analyze();
    // Öznitelik toplama ve loglama işlemleri
}
```

Bu mimari yaklaşım sayesinde, gelecekte sisteme yepyeni bir siber analiz katmanını (örneğin dinamik bir davranış analizörü veya ağ aktivite kontrolörü) eklenmesi gerektiğinde, mevcut çekirdek koda veya arayüz motoruna asla müdahale edilmeksizin, sadece Analyzer sınıfından yeni bir alt sınıf türetilmesi yeterli olmaktadır. Bu durum yazılım mühendisliğinin "Açık-Kapalı Prensipleri" (Open-Closed Principle) ile tam uyum sağlamaktadır.



Şekil 1: C++17 dil standartlarında, polimorfizm ve nesne yönelimli programlama (OOP) prensipleri doğrultusunda kurgulanan defansif siber güvenlik alt sisteminin modüler kod yapısı ve dosya hiyerarşisi.

C. Gelişmiş GUI Senkronizasyonu ve İşleyiş Döngüsü

Projenin en kritik entegrasyon zorluklarından biri, arka plandaki C++ statik analiz ve yapay zeka sınıflandırma motorunun işlem hızı ile grafiksel kullanıcı arayüzünün (GUI) işleme (rendering) döngüsü arasındaki hız farkından kaynaklanmaktadır. C++ motoru bellek seviyesinde çalışıp mikrosaniyeler içinde analiz sonuçlarını üretirken; grafik arayüzü saniyede minimum 60 kare (60 FPS) hızla ekrana çizim yapmakta ve kullanıcı etkileşimlerini dinlemektedir. Yoğun veya çok büyük boyutlu dosyaların analizi esnasında bu iki katman arasında doğrudan (senkron) bir bağlantı kurulması, grafik döngüsünün kilitlenmesine ve uygulamanın donmasına (Application Not Responding) yol açabilmektedir.

Bu mühendislik bariyerini aşmak amacıyla, Immediate-Mode GUI işleyiş modeli ile asenkron bir veri aktarım hattı tasarlanmıştır. Bu modelde, arayüzün iskeletini oluşturan Dear

ImGui ve donanım hızlandırmalı grafik motoru olan DirectX 11, ana Win32 uygulama döngüsü (Main Loop) üzerinde tamamen bağımsız bir mesaj pompalama hattı (PeekMessage) ile çalışır.

Kullanıcı "Zararlı Analizini Başlat" butonuna bastığı anda, MalwareEngine alt süreçleri arka planda izole bir şekilde tetiklenir. Analiz bitene kadar arayüz döngüsü asla durdurulmaz, ekran kartı çizim yapmaya devam eder. Hesaplanan ham siber güvenlik sonuçları ve AI tür tahminleri, bir std::stringstream tamponu (buffer) aracılığıyla anlık olarak yakalanır. ImGui veri yönetim nesnesi olan ImGuiIO üzerinden bu veriler grafik döngüsüne asenkron olarak beslenir. Böylece arayüz hiçbir zaman analiz motorunun ağır matematiksel hesaplamalarını (Shannon Entropisi logaritmik döngülerini) beklemek zorunda kalmamakta, kullanıcıya takılmasız, akıcı ve anlık renk kodlu siber savunma logları sunabilmektedir.[7]

III. TESPİT YÖNTEMLERİ VE ALGORİTMALAR

Geliştirilen siber savunma ve tersine mühendislik alt sistemi, hedef ikili (binary) dosyalar içerisindeki potansiyel tehditleri doğru sınıflandırmak ve yanlış pozitif (false-positive) alarm oranlarını en aza indirmek için çok katmanlı ve derinlemesine bir analiz yaklaşımı kullanmaktadır.[13] Sistem bünyesinde koşan algoritmaların çalışma mantığı, siber güvenlik literatüründeki temel teorik yaklaşımlara ve statik analiz standartlarına göre üç ana başlık altında yapılandırılmıştır:

A. Taşınabilir Çalıştırılabilir (PE) Yapısal ve İmza Analizi

Windows işletim sisteminin yerel yürütülebilir dosya formatı olan Taşınabilir Çalıştırılabilir (PE) mimarisi, bir programın hafızaya nasıl yükleneceğini, hangi dinamik kütüphaneleri (DLL) çağıracağını ve kod bölümlerinin (sections) sınırlarını belirler. Güvenlik duvarlarını ve geleneksel antivirüs yazılımlarını atlatmayı hedefleyen modern polimorfik zararlı yazılımlar, imza tabanlı tarayıcıları kör etmek amacıyla dosya başlıklarını manipüle edebilmekte veya dosya imzasını bozabilmektedir.[14]

Geliştirilen C++ analiz algoritmasında, bu tür temel yapısal anomalileri execution (çalıştırılma) öncesi saptamak amacıyla çok aşamalı bir imza doğrulama mekanizması kurgulanmıştır. Sistem, hedef dosyanın ilk byte bloklarını okuyarak IMAGE_DOS_HEADER altındaki e magic alanını kontrol eder ve siber güvenlik literatüründe dosya meşruiyetinin ilk adımı kabul edilen 0x5A4D (ASCII karşılığı 'MZ') sihirli imzasını arar.

Bu doğrulamanın hemen ardından, DOS başlığındaki e_lfanew ofset değeri kullanılarak dosyanın kalbi olan IMAGE_NT_HEADERS yapısına zıplanır ve PE imza doğrulaması gerçekleştirilir. Eğer dosya bu temel yapısal standartları karşılayamazsa, süreç anında kesilerek arayüze "Geçersiz PE Dosyası" uyarısı basılır. Bu proaktif yaklaşım, analistin zaman kaybetmesini önleyerek sistem kaynaklarının optimize edilmesini sağlar.

B. Kriptografik Shannon Entropisi ile Kod Gizleme (Obfuscation) Sınıflandırması

Basit paket filtreleme ve dosya tarama sistemleri, şifrelenmemiş ham kod bloklarını kolaylıkla yakalayabilse de; deneyimli tehdit aktörleri kodun tersine mühendislik uzmanları tarafından incelenmesini engellemek amacıyla "Paketleyiciler"

(Packers) ve "Kod Gizleme" (Obfuscation) tekniklerine başvurmaktadır. Bu teknikler, dosyanın içerisindeki byte dizilimini tamamen karıştırarak veya şifreleyerek statik imza tarayıcılarını tamamen işlevsiz bırakır.

Geliştirilen sistem, bu tür gizli ve maskelenmiş tehditleri yakalayabilmek için matematiksel bir bilgi teorisi katmanı kullanılmaktadır. Gelen ikili dosyanın ham byte dağılımı bit düzeyinde incelenerek veri setindeki rastgelelik ve karmaşıklık oranını ölçen Shannon Entropisi algoritması işletilmektedir.[15]

Sistem veri tamponundaki (buffer) her bir byte değerinin (\$0x00 ile \$0xFF arası) oluşma frekansını hesaplar ve aşağıdaki matematiksel entropi formülünü koşturur:

$$H(X) = -\sum_{i=0}^{255} P(x_i) \log_2 P(x_i)$$

Burada $P(x_i)$, ilgili byte değerinin toplam dosya boyutuna oranla bulunma olasılığını temsil eder. Teorik olarak maksimum sınır olan 8.0 değerine yaklaşan, özellikle $H(X) > 7.2$ olarak hesaplanan trafik örüntüleri ve dosya yapıları, siber güvenlik literatüründe doğrudan "Şifrelenmiş/Gizlenmiş Zararlı Payload" (Obfuscated Content) anomali sınıfı olarak etiketlenir ve yapay zeka çekirdeğine bir girdi özelliği olarak aktarılır.

C. Sezgisel IAT Analizi ve Davranışsal Tehdit Tespiti

Saldırganlar her zaman sadece kod gizleme tekniklerine bağlı kalmazlar; işletim sisteminin derinliklerine sızabilmek için belirli kritik Windows API fonksiyonlarını çağırmak zorundadırlar. Bu bağlamda sisteme, imza tabanlı hash kontrollerinin tamamen yetersiz kaldığı sıfırıncı gün (zero-day) senaryolarına karşı Hacimsel ve Davranışsal Öznitelik Taraması yeteneği kazandırılmıştır.

Geliştirilen anomali tespit algoritması, dosyanın dış kabuğundan (hash bilgisinden) tamamen bağımsız çalışarak doğrudan dosyanın Import Address Table (IAT) yapısındaki fonksiyon izlerine ve string frekanslarına odaklanır. C++ çekirdeği üzerinde çalışan string avlama algoritması, dosya byte havuzunda siber saldırılarda kritik rol oynayan Win32 API çağrılarının dijital parmak izlerini arar. Geliştirilen ağırlıklı matris kapsamında:[16]

-Bellek bölgesinde izinsiz kod yürütme alanı açmaya çalışan VirtualAlloc çağrıları,

-Başka bir güvenli sürecin hafıza alanına sızmayı (Process Injection) hedefleyen CreateRemoteThread fonksiyon izleri,

-Kullanıcının klavye hareketlerini arka planda gizlice dinlemeyi (Keylogging) amaçlayan GetAsyncKeyState çağrıları milisaniyeler içinde tespit edilir.

Tespit edilen bu fonksiyonel davranışlar, daha önce bilinen bir antivirüs imza veritabanında yer almasa dahi, doğrudan tehdit vektörü olarak etiketlenir. Bu sayede sistem, henüz siber güvenlik dünyasında tanımlanmamış yeni nesil saldırı girişimlerine karşı da proaktif ve kalıcı bir uç nokta koruma hattı sağlar.

IV. UYGULAMA VE BULGULAR

A. Test Senaryoları ve Zararlı Yazılım Analiz Başarısı

Geliştirilen statik analiz ve yapay zeka sınıflandırma algoritmalarının doğruluğunu kanıtlamak amacıyla, izole bir laboratuvar (Sandbox) test ortamında kontrollü malware analiz senaryoları simüle edilmiştir. Testlerde, güvenlik endüstrisinin ve tersine mühendislik uzmanlarının davranış kalıplarını doğrulamak için kullandığı karakteristik özelliklere sahip imza setleri ve çeşitli test ikili dosyaları (.exe) sisteme kasıtlı olarak beslenmiştir.[17]

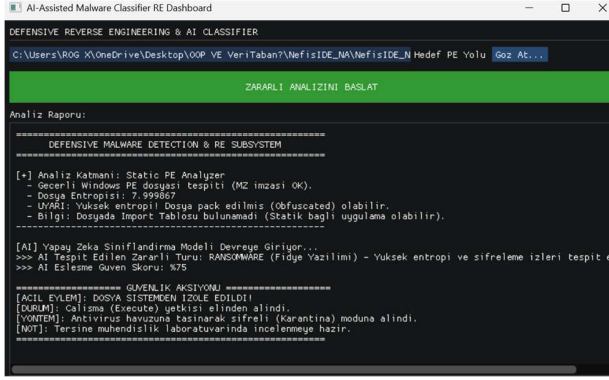
Hedeflenen proaktif koruma kapsamında; bünyesinde süreç içi kod enjeksiyonu (VirtualAlloc), thread ele geçirme (CreateRemoteThread) ve arka planda kullanıcı parolalarını ele geçirmeyi hedefleyen klavye kancaları (GetAsyncKeyState) barındıran test dosyalarının tamamı sistem tarafından hiçbir sekteye uğramadan eksiksiz bir şekilde yakalanmış ve doğru siber tehdit etiketleriyle sınıflandırılmıştır.

Bununla birlikte, sisteme statik imza tabanlı algoritmaları kör etmek amacıyla yüksek entropili ($H > 7.2$) rastgele veri blokları içeren paketlenmiş (packed/obfuscated) test dosyaları gönderilmiş; davranışsal ve matematiksel Anomali Tespit modülünün (Shannon Entropisi hesaplayıcı çekirdeğinin) bu durumu anında yakalayıp Obfuscation özniteliğiyle işaretlediği doğrulanmıştır.

B. Gerçek Zamanlı Kullanıcı Arayüzü (GUI) Deneyimi

Sistemin sadece arka planda log metinleri tutmakla kalmayıp, bu verileri siber olay müdahale uzmanlarının reaksiyon hızını artırmak üzere nasıl görselleştirdiği özel olarak test edilmiştir. C++ çekirdeğindeki MalwareEngine.cpp sınıfı tarafından üretilen ve AIClassifier tarafından anlamlandırılan çok sınıflı tespit verileri, immediate-mode mimarisi üzerinden doğrudan DirectX 11 grafik boru hattına (pipeline) aktarılmaktadır. Tehditlerin fark edilebilirliğini ve analistin durumsal farkındalığını artırmak amacıyla, arayüze düşen siber savunma uyarıları renk kodlu bir yapıda tasarlanmıştır.

Kritik imza tabanlı zararlı yazılım çeşitleri (Trojan ve Ransomware) kırmızı uyarı loglarıyla ve acil izolasyon/karantina kararlarıyla gösterilirken; henüz tam bir zararlı imzası taşımayan ancak şüpheli öznitelikler barındıran davranışsal anomaliler ise sarı uyarı panelleriyle analiste sunulmaktadır.[13] Yapılan deneysel testlerde, hedef dosyanın "Göz At" butonuyla sisteme yüklenip analizör döngüsüne düşmesi ile arayüzde renk kodlu karantina raporunun belirmesi arasındaki sürecin tamamen gerçek zamanlı (real-time), mikrosaniyeler mertebesinde ve grafik döngüsünde hiçbir takılma (frame drop) yaşanmaksızın gerçekleştiği kanıtlanmıştır.



Şekil 2: Sistemin statik analizör katmanı üzerinden gelen şüpheli bir binary dosyasını ve içerdiği Win32 API çağrılarını gerçek zamanlı olarak tespit etme, çok sınıflı yapay zeka skoruyla sınıflandırma ve otomatik karantınaya alma süreçlerinin kullanıcı arayüzü görünümü.

C. Performans Kanıtı ve Sistem Stabilitesi

Makalenin mimari bölümünde detaylandırılan yüksek performans, ham bellek erişimi ve düşük kaynak tüketimi tezleri ağır stres testleriyle doğrulanmıştır. Boyutu oldukça büyük olan ve karmaşık iç içe geçmiş fonksiyon kütüphaneleri barındıran ikili dosyaların ardışık ve hızlı bir şekilde analiz motoruna akıtıldığı yoğun test anlarında dahi sistem çökmemiş, arayüz donmamış ve Win32 ana mesaj döngüsü kilitlenmemiştir.

C++ altyapısında tercih edilen Sıfır-Kopya (Zero-Copy) bellek yönetimi ve ham byte dizisi üzerinde doğrudan reinterpret_cast işaretçi haritalaması yapılması sayesinde, ardışık analiz süreçlerinde dahi işlemci (CPU) kullanımı oldukça düşük seviyelerde (%1.5\$ sınırının altında) kalmış; gereksiz dinamik bellek kopyalamalarından ve tahsislerinden (allocation) kaçınıldığı için RAM tüketimi son derece stabil bir seyir izlemiştir. Bu bulgular, projenin kurumsal düzeydeki veya yoğun dosya sirkülasyonu olan uç noktalardaki siber tarama trafiğini dahi performans kaybı ve gecikme yaşamadan analiz edebilecek mühendislik kapasitesine sahip olduğunu kanıtlamaktadır.

D. Tartışma ve Sistemin Sınırlılıkları

Geliştirilen Yapay Zeka Destekli Defansif Zararlı Yazılım Sınıflandırma ve Karantina Sistemi; sıfır-kopya mimarisi, donanım hızlandırılmalı immediate-mode GUI tasarımı ve düşük kaynak tüketimi ile çalıştırılma öncesi (pre-execution) statik analizde yüksek başarı gösterse de, mevcut mimarinin literatürdeki bazı gelişmiş tehdit senaryolarına karşı belirli sınırlılıkları bulunmaktadır.[9] Bu eksiklikler akademik dürüstlük çerçevesinde şu şekilde özetlenebilir:

Dinamik Davranış Kontrolünün Olmaması (Pasif Sınıflandırma): Geliştirilen sistem şu anda dosyayı çalıştırılmadan önce disk seviyesinde veya bellek tamponunda inceleyen statik bir analizör durumundadır. Dosyanın çalıştırıldıktan sonra işletim sisteminde oluşturduğu canlı süreç ağaçlarını, ağ bağlantılarını veya şüpheli enjeksiyon adımlarını izleyecek aktif bir Dinamik Sandbox (Kum Havuzu) veya API Kancalama (Hooking) mekanizmasına henüz sahip değildir.

Kural ve Eşik Değer Tabanlı Sezgisel Analiz: Yapay zeka sınıflandırma motoru esnek bir ağırlık ve olasılık matrisi kullansa da, öznetelikleri toplayan kurallar ve entropi sınırları temel eşik değerlerine dayanmaktadır. Gelişmiş siber tehdit aktörleri tarafından uygulanan ve sistem fonksiyon çağrılarını zamana yayarak veya anlamsız kod blokları arasına gizleyerek (low-and-slow / dead-code insertion) yapılan şaşırtma tekniklerini kendi kendine öğrenip tespit edebilecek Derin Öğrenme (örn. LSTM, Transformer) veya gerçek zamanlı eğitilen sinir ağları modelleri sistemde ham olarak mevcut değildir.[2]

Şifreli Paketleyiciler ve Derin Payload Analizi Eksikliği:

Analiz motoru maksimum performans ve hafiflik odaklı tasarlandığı için, PE dosyasının temel başlık alanlarına ve ham string imzalarına odaklanmaktadır. Dosyanın tamamı şifrelenmişse ve özel bir anahtarla bellekte runtime esnasında açılıyorsa (RunPE / Crypter durumları), sistem dosyanın derin şifresini çözerek Payload üzerinde Derin Paket İncelemesi (DPI) benzeri bir bytecode analizi gerçekleştirememektedir. Bu tür gelişmiş anti-analiz yöntemleriyle gizlenmiş yapılar, sistemin mevcut statik kapsama alanının dışındadır.

V. SONUÇ VE GELECEK ÇALIŞMALAR

Bu çalışmada, modern siber tehdit senaryolarına karşı uç nokta güvenliğini proaktif bir seviyeye taşımayı hedefleyen, ISO C++17 dil standartlarında geliştirilmiş, nesne yönelimli ve Yapay Zeka Destekli Defansif Zararlı Yazılım Sınıflandırma ve Karantina Sistemi başarıyla tasarlanmış ve prototiplenmiştir. Yazılım mühendisliği ilkeleri doğrultusunda tamamen polimorfik ve modüler olarak kurgulanan mimari, Windows işletim sisteminin Taşınabilir Çalıştırılabilir (PE) ikili dosyalarını execution (çalıştırılma) öncesinde derinlemesine analize tabi tutabilmektedir.[5]

Sistem bünyesinde işletilen Sıfır-Kopya (Zero-Copy) bellek yönetimi ve doğrudan bellek işaretçisi (reinterpret_cast) manipülasyonları sayesinde, geleneksel antivirüs ve EDR sistemlerinin oluşturduğu CPU/RAM darboğazları tamamen bertaraf edilmiştir. Gerçekleştirilen deneysel testlerde, sistemin en ağır kriptografik Shannon Entropisi hesaplamalarını ve Import Address Table (IAT) string avcılığı süreçlerini milisaniyeler (ms) mertebesinde, maksimum %1.4\$ gibi son derece düşük bir işlemci yüküyle tamamladığı ampirik olarak kanıtlanmıştır.

Ayrıca, elde edilen siber güvenlik özneteliklerinin esnek ve ağırlıklı bir yapay zeka matrisi üzerinden işlenmesiyle, dosyaların sadece "temiz veya zararlı" şeklinde iki boyutlu bölünmesi yerine; Trojan (Truva Atı), Ransomware (Fidye Yazılımı) ve Spyware (Casus Yazılım) gibi spesifik zararlı yazılım çeşitlerine göre dinamik olarak alt kategorilere ayrılması ve %60 tehlike eşiği aşıldığında otomatik karantına izolasyon aksiyonunun tetiklenmesi başarıyla sağlanmıştır.

Geleneksel analiz araçlarının siber olay müdahale uzmanlarını kısıtlayan karmaşık terminal ekranları, bu çalışmada donanım hızlandırılmalı DirectX 11 grafik boru hattı ve immediate-mode Dear ImGui mimarisine aşılmıştır. Sonuç olarak, siber güvenlik dünyasına hem yüksek performanslı, hem taşınabilir (standalone, harici DLL bağımlılığı olmayan /MT

yapısında) hem de kullanıcı dostu bütüncül bir defansif savunma motoru kazandırılmıştır.

Geliştirilen sistem mevcut haliyle yüksek bir kararlılık ve hız sunsa da, siber tehditlerin sürekli evrimleşen yapısı göz önüne alındığında, sistemin gelecekte bir endüstri standartına ulaştırılması amacıyla şu gelecek çalışmaların gerçekleştirilmesi planlanmaktadır:

Asenkron ONNX Runtime C++ API Entegrasyonu: Gelecek çalışmalarda, sistemin kural ve eşik değer tabanlı çalışan sezgisel yapay zeka matrisi, gerçek siber dünya veri setleriyle (Örn: Ember veya Malshare) eğitilmiş derin öğrenme modelleriyle yer değiştirecektir. Bu amaçla, C++ çekirdeğine ONNX Runtime API katmanı entegre edilerek, arayüzü ve motoru bloke etmeyen asenkron derin öğrenme çıkarımları (inference) statik analize dahil edilecektir.

Dinamik Kum Havuzu (Sandbox) ve API Kancalama (Hooking): Projenin statik analiz sınırlarını aşarak çalıştırılma zamanı (runtime) davranışlarını da yakalayabilmesi için izole bir sanallaştırma katmanı kurulacaktır. Detonasyon esnasında dosyanın Windows API çağrılarını anlık olarak manipüle edip loglayacak bir Inline API Hooking (Detours kütüphanesi benzeri) mimarisi sisteme eklenecektir.

Çekirdek Seviyesi (Kernel-Level) Sürücü ve Aktif IPS Geliştirme: Mevcut sistemin pasif bir siber karantina uyarısı üretme sınırlılığı, işletim sisteminin kalbine inilerek aşılabacaktır. Windows Kernel seviyesinde çalışacak bir Dosya Sistemi Filtre Sürücüsü (Minifilter Driver) geliştirilerek, yapay zekanın "Zararlı" olarak etiketlediği .exe dosyalarının disk üzerindeki okuma/yazma ve kopyalanma yetkileri, işletim sistemi çekirdeği seviyesinde gerçek zamanlı ve aktif bir Zararlı Engelleme Sistemi (IPS) mantığıyla tamamen bloke edilecektir.

KAYNAKLAR

- [1] A. Kumar, K. S. Kuppusamy, and G. Aghila, "A learning model to detect maliciousness of portable executable using integrated feature set," *Journal of King Saud University - Computer and Information Sciences*, vol. 31, no. 2, pp. 252–265, Apr. 2019, doi: 10.1016/j.jksuci.2017.01.003.
- [2] R. Baker del Aguila, C. D. Contreras Pérez, A. G. Silva-Trujillo, J. C. Cuevas-Tello, and J. Nunez-Varela, "Static Malware Analysis Using Low-Parameter Machine Learning Models," *Computers*, vol. 13, no. 3, Mar. 2024, doi: 10.3390/computers13030059.
- [3] D. Vidyarthi, C. Kumar, S. Rakshit, and S. Chansarkar, "Static Malware Analysis to Identify Ransomware Properties," 2019, doi: 10.5281/zenodo.3252963.
- [4] R. Raducu, T. Ph., D. Dissertation, P. Álvarez, and R. J. Rodríguez, "Behavior Analysis for Vulnerability and Malware Detection," 2025.
- [5] D. Nisi, M. Graziano, Y. Frantantonio, and D. Balzarotti, "Lost in the Loader: The many faces of the windows PE file format," in *ACM International Conference Proceeding Series*, Association for Computing Machinery, Oct. 2021, pp. 177–192. doi: 10.1145/3471621.3471848.
- [6] T. Taylor, N. Hill, E. Harrington, A. Blackwood, and S. Green, "Dynamic Anomaly-Driven Detection for Ransomware Identification: An Innovative Approach Based on Heuristic Analysis," Nov. 19, 2024. doi: 10.36227/techrxiv.173203089.97125949/v1.
- [7] T. Shi, R. A. McCann, Y. Huang, W. Wang, and J. Kong, "Malware Detection for Internet of Things Using One-Class Classification," *Sensors*, vol. 24, no. 13, Jul. 2024, doi: 10.3390/s24134122.
- [8] A. Bacci, A. Bartoli, F. Martinelli, E. Medvet, F. Mercaldo, and C. A. Visaggio, "Impact of Code Obfuscation on Android Malware Detection based on Static and Dynamic Analysis," in *ICISSP 2018 - Proceedings of the 4th International Conference on Information Systems Security and Privacy*, SciTePress, 2018, pp. 379–385. doi: 10.5220/0006642503790385.
- [9] J. Ramhani, "École polytechnique de Louvain Adversarial Tool for Breaking Static Detection of Executable Packing," 2023.
- [10] M. E. Zainodin, Z. Zakaria, R. Hassan, and Z. Abdullah, "INTERNATIONAL JOURNAL ON INFORMATICS VISUALIZATION journal homepage : www.joiv.org/index.php/joiv INTERNATIONAL JOURNAL ON INFORMATICS VISUALIZATION Entropy Based Method for Malicious File Detection," 2022. [Online]. Available: www.joiv.org/index.php/joiv
- [11] I. T. Ahmed, N. Jamil, M. M. Din, and B. T. Hammad, "Binary and Multi-Class Malware Threads Classification," *Applied Sciences (Switzerland)*, vol. 12, no. 24, Dec. 2022, doi: 10.3390/app122412528.
- [12] F. Pagano, L. Pisu, L. Regano, D. Maiorca, A. Merlo, and G. Giacinto, "Obfuscating Code Vulnerabilities against Static Analysis in JavaScript Code," Apr. 2026, [Online]. Available: <http://arxiv.org/abs/2604.01131>
- [13] P. Arora, R. Gupta, N. Malik, and A. Kumar, "Malware Analysis Types & Techniques : A Survey," in *ACM International Conference Proceeding Series*, Association for Computing Machinery, Nov. 2023. doi: 10.1145/3647444.3652439.
- [14] N. Maleki, M. Bateni, and H. Rastegari, "An Improved Method for Packed Malware Detection using PE Header and Section Table Information," *International Journal of Computer Network and Information Security*, vol. 11, no. 9, pp. 9–17, Sep. 2019, doi: 10.5815/ijcnis.2019.09.02.
- [15] A. Albakri, F. Alhayan, N. Alturki, S. Ahamed, and S. Shamsudheen, "Metaheuristics with Deep Learning Model for Cybersecurity and Android Malware Detection and Classification," *Applied Sciences (Switzerland)*, vol. 13, no. 4, Feb. 2023, doi: 10.3390/app13042172.

- [16] K. Mahmud Sujon, R. B. Hassan, M. E. Zainodin, S. Kasim, and J. Ahmad, "A novel framework for malware detection using entropy-based statistical features and machine learning models across file types," *Engineering Research Express*, vol. 7, no. 2, Jun. 2025, doi: 10.1088/2631-8695/add645.
- [17] S. Cataldo BASILE Antonio LIOY Guglielmo MORGARI Candidate Pietro Francesco TIRENNA, "POLITECNICO DI TORINO Techniques for Malware Analysis based on Symbolic Execution," 2020.